Авторизация и разграничение доступа

RBAC, проверка владельца ресурса, 401/403 и защита REST API

Результаты занятия

- Различать Authentication и Authorization
- Понимать принцип deny by default
- Понимать принцип least privilege
- Различать 401 Unauthorized и 403 Forbidden
- Реализовывать Role-Based Access Control
- Создавать middleware requireRole()
- Защищать административные CRUD endpoints
- Проверять владельца конкретного ресурса
- Не доверять userId и role из HTTP body
- Выполнять authorization check на каждом защищённом запросе

Учебные вопросы

1	**Authentication vs Authorization**	Кто пользователь и что ему разрешено
2	**RBAC**	Роли user/admin и middleware requireRole
3	**Защита REST endpoints**	401, 403 и server-side access control
4	**Доступ к конкретным объектам**	Проверка владельца Order и защита от IDOR
5	**Безопасная архитектура**	Least privilege, deny by default и проверка каждого запроса

Учебный вопрос 1. Authentication vs Authorization

Сначала устанавливаем identity, затем принимаем решение о доступе

Authentication и Authorization

Authentication — AuthN

- Кто отправил запрос?
- Проверяет identity
- Login и JWT
- requireAuth
- Результат: req.user

Authorization — AuthZ

- Можно ли этому User выполнить действие?
- Проверяет role, ownership и другие правила
- requireRole или проверка в Service
- Выполняется после Authentication
- Результат: allow или deny

Аутентифицированный пользователь не получает доступ ко всему

- User может успешно пройти login и иметь валидный JWT.
- Это подтверждает identity, но не даёт автоматически административные права.
- Покупатель может просматривать свои заказы.
- Admin может управлять каталогом товаров.
- Каждое чувствительное действие требует отдельного authorization decision.

Примеры правил Internet Shop

Действие	Гость	User	Admin
GET /api/products	Разрешено	Разрешено	Разрешено
POST /api/products	Запрещено	Запрещено	Разрешено
PATCH /api/products/:id	Запрещено	Запрещено	Разрешено
DELETE /api/products/:id	Запрещено	Запрещено	Разрешено
GET /api/orders/my	Запрещено	Разрешено	Разрешено
GET чужого заказа	Запрещено	Запрещено	Зависит от политики приложения

Authentication → Authorization → Resource

От подтверждения личности к проверке прав доступа

Internet Shop
Безопасный доступ к ресурсам вашего магазина

1 Клиент
Отправляет запрос



Например:

```
POST /api/orders
Authorization: Bearer <JWT>

{
  "productId": 123,
  "quantity": 2
}
```

2 Authentication
Кто вы?



- Проверка JWT
- Валидация подписи
- Проверка срока действия
- Извлечение данных пользователя

Результат:
пользователь аутентифицирован (`req.user`)

3 Authorization
Можно ли?



- Проверка роли (role)
- Проверка прав (`permissions`)
- Проверка владения объектом (`ownership`)
- Бизнес-правила

Результат:
доступ разрешён или запрещён

4 Resource
Доступ к ресурсу



Доступ к защищённому ресурсу:

- данные пользователя
- заказы
- товары
- и т.д.

Доступ разрешён
Возвращаем данные ресурса (JSON)

```
{
  "id": 15,
  "product": "Laptop",
  "price": 999
}
```

Доступ запрещён
Возвращаем ошибку

```
{
  "error": "Forbidden",
  "message": "Недостаточно
             прав доступа"
}
```

Ключевые отличия

1 Authentication
Отвечает на вопрос:
«Кто вы?»

Подтверждает личность пользователя (JWT, логин/пароль).

2 Authorization
Отвечает на вопрос:
«Можно ли?»

Проверяет права, роли, владение объектом.

Resource
Это то, к чему есть **или нет доступа.**

Данные, API, файлы, функции и т.д.

Пример из Internet Shop

1. Клиент отправляет запрос с JWT.
2. **requireAuth** проверяет токен и получает `req.user`.
3. **requireRole** проверяет роль (admin / user).
4. Если нужно — проверяется владелец объекта (например, **order.id + req.user.id**).
5. Доступ к ресурсу (данные заказа, товара и т.д.).

Важно помнить

- **401** — нет действительных аутентификационных данных.
- **403** — пользователь аутентифицирован, но нет прав.
- Проверяйте права на объект при каждом запросе (OWASP).
- Не доверяйте **userId** из запроса (body) — используйте **req.user.id**.

Deny by default

- Если правило явно не разрешает действие, доступ должен быть запрещён.
- Новый endpoint не должен случайно оказаться доступным всем.
- Разрешения добавляются явно.
- Middleware и Service должны иметь понятные правила доступа.
- Безопасная политика начинается с запрета, а не с разрешения.

Least privilege

- Пользователь и сервис должны получать только минимально необходимые права.
- Обычный User не должен иметь административные права.
- Admin получает дополнительные разрешения только для нужных операций.
- JWT должен содержать только необходимые authorization claims.
- Frontend не определяет реальные права пользователя.

Claims — утверждения (данные) внутри JWT-токена, например идентификатор пользователя (sub), его роль (role) и срок действия токена (exp).

Учебный вопрос 2. Role-Based Access Control

RBAC принимает решение на основании роли пользователя

Role-Based Access Control

- RBAC — модель, в которой разрешения назначаются ролям, а пользователю назначается одна или несколько ролей.
- Для учебного Internet Shop используем роли user и admin.
- user — обычный покупатель.
- admin — управление каталогом и административные операции.
- Роль хранится в User и включается в access JWT.
- Authorization middleware сравнивает роль с разрешёнными ролями.

Role в User и JWT

PRISMA

```
model User {
  id           Int      @id @default(autoincrement())
  name         String
  email        String   @unique
  passwordHash String   @map("password_hash")
  role         String   @default("user")
  orders       Order[]

  @@map("users")
}
```

Что важно

- По умолчанию новый пользователь получает role = user.
- Регистрационный endpoint не должен принимать role из req.body.
- Admin назначается доверенной административной процедурой.

Почему нельзя принимать role при регистрации

- HTTP body полностью контролируется клиентом.
- Атакующий может отправить role: admin.
- Если сервер доверится этому значению, возникнет vertical privilege escalation.
- Публичная регистрация должна сама назначать безопасную роль user.
- Повышение роли выполняется только доверенным серверным механизмом.

Опасный register request

Запрос: POST /api/auth/register

JSON

```
{
  "name": "Attacker",
  "email": "attacker@example.com",
  "password": "Example123!",
  "role": "admin"
}
```

Результат

Сервер игнорирует role из body и создаёт пользователя с role = user.

Пояснение

- Authorization attributes нельзя доверять клиентскому вводу.

JWT содержит role

```javascript
return jwt.sign(
  { role: user.role },
  process.env.JWT_SECRET,
  {
    algorithm: "HS256",
    subject: String(user.id),
    issuer: process.env.JWT_ISSUER,
    audience: process.env.JWT_AUDIENCE,
    expiresIn: process.env.JWT_EXPIRES_IN
  }
);
```

Что важно

- sub идентифицирует User.
- role используется для простого RBAC.
- Claims становятся доверенными только после jwt.verify().
- Роль в JWT является snapshot на момент выпуска token.

Role в JWT может устареть

- Если роль User изменилась в PostgreSQL, уже выданный JWT не меняется.
- Старая роль может действовать до истечения срока (expiration) token.
- Короткий срок жизни access token уменьшает это окно.
- Для критичных действий можно дополнительно загрузить актуальный User из базы.
- После изменения роли пользователю целесообразно получить новый access token.

requireRole()

```javascript
export function requireRole(...allowedRoles) {
  return function (req, res, next) {
    if (!req.user) {
      return res.status(401).json({
        error: "Authentication required"
      });
    }

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        error: "Forbidden"
      });
    }

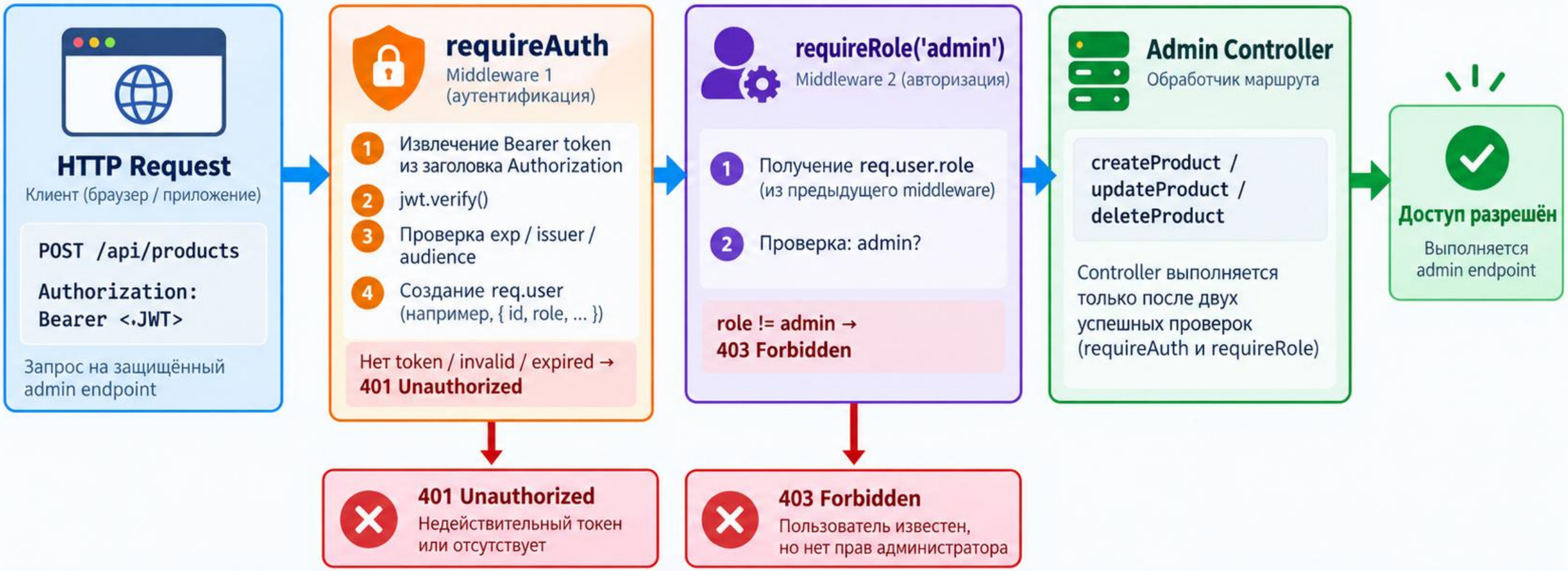
    next();
  };
}
```

Что важно

- Middleware предполагает, что requireAuth уже создал req.user.
- allowedRoles определяет допустимые роли.
- Недостаточная роль приводит к 403.
- При успехе запрос передаётся Controller.

requireAuth → requireRole(admin) → Admin Controller

Цепочка middleware в Express: сначала аутентификация, затем проверка роли, затем контроллер



1. **requireAuth** подтверждает, кто пользователь
2. **requireRole(admin)** проверяет право доступа
3. Только затем запускается Admin Controller

! **401** = нет действительной аутентификации

! **403** = пользователь известен, но прав недостаточно

Учебный вопрос 3. Защита REST endpoints

Access control должен выполняться на сервере для каждого запроса

401 и 403

Status	Когда использовать	Пример
401 Unauthorized	Нет действительных authentication credentials	JWT отсутствует, неверен или expired (просрочен)
403 Forbidden	Identity известна, но прав недостаточно	User пытается вызвать admin endpoint

Frontend не является механизмом безопасности

- Можно скрыть кнопку Delete для обычного User.
- Но пользователь может вручную отправить DELETE через DevTools, curl или Postman.
- Реальная проверка должна выполняться backend.
- UI-проверка нужна для удобства, server-side authorization — для безопасности.
- Каждый защищённый endpoint должен проверять доступ независимо.

Admin-only Product routes

```javascript
router.post(
  "/",
  requireAuth,
  requireRole("admin"),
  createProduct
);

router.patch(
  "/:id",
  requireAuth,
  requireRole("admin"),
  updateProduct
);

router.delete(
  "/:id",
  requireAuth,
  requireRole("admin"),
  deleteProduct
);
```

Что важно

- requireAuth выполняется первым.
- requireRole проверяет admin.
- Controller запускается только после обеих проверок.
- GET каталога можно оставить публичным.

Что происходит при DELETE /api/products/15

1	**Проверка JWT**	requireAuth подтверждает identity.
2	**Проверка role**	requireRole проверяет admin.
3	**Controller**	Получает запрос только при разрешённом доступе.
4	**Service**	Выполняет бизнес-логику удаления.
5	**PostgreSQL**	Изменение выполняется только после всех проверок.

Безопасные API — надёжные приложения

401 vs 403 в защищённом REST API

Один API. Разные причины. Понимай разницу — пиши безопасные приложения.

Express.js

401 Unauthorized

Сервер не получил действительные данные аутентификации

Клиент

HTTP запрос к защищённому endpoint

```
POST /api/products
Authorization: ?
Content-Type: application/json
...
```

Express.js

requireAuth
Проверяет JWT в заголовке Authorization

JWT невалиден или отсутствует

Контроллер (не выполняется)

Ответ сервера

```
HTTP/1.1 401 Unauthorized

{
  "error": "Unauthorized",
  "message": "Invalid or
   missing token"
}
```

Доступ отклонён (401)

Возможные случаи:

- **Нет заголовка Authorization**
- **Невалидный Bearer token** (неверная подпись, некорректный формат)
- **Просроченный JWT** (expired token)

403 Forbidden

Пользователь известен, но прав недостаточно

Клиент

HTTP запрос к защищённому endpoint

```
DELETE /api/products/15
Authorization: Bearer eyJhbGci...
Content-Type: application/json
...
```

Передан валидный Bearer JWT
Пользователь успешно аутентифицирован

Express.js

requireAuth
Проверяет JWT
Создаёт req.user

Аутентификация успешна

requireRole('admin')
Проверяет роль пользователя

Недостаточно прав (например, роль user)

Контроллер (не выполняется)

Ответ сервера

```
HTTP/1.1 403 Forbidden

{
  "error": "Forbidden",
  "message": "Access denied.
   Admin role required."
}
```

Доступ запрещён (403)

Краткое сравнение

Критерий	401 Unauthorized	403 Forbidden
Смысл	Нет действительной аутентификации	Аутентификация есть, но доступ запрещён
Причина	Обычно связан с отсутствующим / неверным / просроченным JWT	Обычно связан с недостаточной ролью или правами
Кто пользователь?	Неизвестен (не прошёл аутентификацию)	Известен (прошёл аутентификацию)
Где возникает?	На этапе **requireAuth** (до контроллера)	На этапе проверки авторизации (например, `requireRole('admin')`)

Сначала аутентификация (КТО ты?), затем авторизация (ЧТО тебе можно?).

Используй 401 и 403 по назначению — это делает API понятным, безопасным и удобным.

User и Admin

user

- Просмотр каталога
- Создание собственного заказа
- Просмотр собственных заказов
- Нет управления Product
- Нет управления чужими пользователями

admin

- Все разрешения обычного пользователя по выбранной политике
- Создание Product
- Изменение Product
- Удаление Product
- Административные функции

Не дублируйте authorization logic хаотично

- Role checks для маршрутов удобно централизовать в middleware.
- Object ownership часто проверяется в Service вместе с запросом данных.
- Одинаковые правила должны использовать общие функции.
- Разрозненные if-проверки легче забыть при добавлении нового endpoint.

Учебный вопрос 4. Доступ к конкретным объектам

Role недостаточно: нужно проверять, кому принадлежит конкретный Order

Проблема IDOR / object-level authorization

- Пользователь имеет право читать свои Order.
- Это не означает право читать любой Order.
- Если endpoint использует только req.params.id, клиент может подставить другой id.
- Предсказуемый id не является механизмом защиты.
- Backend должен проверять связь ресурса с текущим req.user.id.

Опасный endpoint без ownership check

Запрос: GET /api/orders/101

JAVASCRIPT

```
const order = await prisma.order.findUnique({
  where: { id: Number(req.params.id) }
});
```

Результат

Если Order 101 принадлежит другому User, данные всё равно могут быть возвращены.

Пояснение

- Проверка только id ресурса недостаточна.

Безопасный запрос: id + userId

```javascript
export async function findOwnOrder(orderId, userId) {
  return prisma.order.findFirst({
    where: {
      id: orderId,
      userId
    },
    include: {
      items: {
        include: { product: true }
      }
    }
  });
}
```

Что важно

- userId берётся из req.user.id.
- В запрос одновременно входит id заказа и его владелец.
- Чужой Order не будет возвращён.
- Так ownership становится частью запроса к данным.

Controller использует req.user.id

```javascript
export async function getMyOrder(req, res, next) {
  try {
    const orderId = Number(req.params.id);
    const order = await findOwnOrder(
      orderId,
      req.user.id
    );

    if (!order) {
      return res.status(404).json({
        error: "Order not found"
      });
    }

    res.json(order);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Identity приходит из проверенного JWT.
- Клиент не выбирает userId.
- Отсутствующий или чужой заказ можно одинаково представить как 404.
- Это уменьшает раскрытие существования чужих ресурсов.

Object-level authorization: Order.id + req.user.id

Доступ к конкретному заказу разрешается только если совпадают **И Order.id, И req.user.id**

1. HTTP-запрос от клиента

Браузер или HTTP-клиент



```
GET /api/orders/my/15 HTTP/1.1
Host: example.com
Authorization: Bearer <JWT>
...
```

Клиент запрашивает свой заказ с ID 15, передавая JWT в заголовке Authorization.

2. Express middleware: requireAuth

Проверяет JWT и создаёт **req.user**

```
// middleware/requireAuth.js
const requireAuth = (req, res, next) => {
  const payload = verifyToken(req);
  req.user = {
    id: payload.id,  // из JWT
    ..
  };
  next();
};
```

req.user.id = 5

req.user.id берётся из проверенного JWT.

3. Контроллер → Сервис

Передаём два значения: orderId и userId

```
// controller
const orderId = Number(req.params.id);
const userId = req.user.id; // 5
return orderService.findMyOrder(
    orderId,   // 15
    userId     // 5
));
```

В сервис передаются:

orderId = 15 **userId = 5**

4. Prisma / PostgreSQL

Ищем заказ по двум условиям сразу

```
// service (Prisma)
const order = await prisma.order.findUnique({
  where: {
    id: orderId,    // 15
    userId: userId, // 5
  }
});
```

Prisma

PostgreSQL

Безопасный запрос: Order.id + req.user.id

Проверяется не только Order.id, но и владелец заказа.

Примеры данных в таблице orders

id	userId	status	createdAt
15	5	PAID	2024-06-10
15	8	PAID	2024-06-11
16	5	NEW	2024-06-12

- Совпадает пара (15, 5) — **доступ разрешён**
- Тот же заказ, но другой владелец (15, 8) — доступ **запрещён** (не найдено)
- Другой заказ (16, 5) — не совпадает с запросом

Заказ найден (свой заказ)

Возвращаем данные заказа (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "id": 15,
  "userId": 5,
  "status": "PAID",
  "total": 12500,
  "createdAt": "2024-06-10T10:00:00Z"
}
```

Заказ не найден

Нет заказа с такой парой (id, userId). Возвращаем 404, чтобы не раскрывать чужие заказы.

```
HTTP/1.1 404 Not Found
Content-Type: application/json
{
  "message": "Order not found"
}
```

Главные выводы:

1. Проверяется не только Order.id, но и владелец заказа.
2. **req.user.id** берётся из проверенного JWT.
3. Клиент не может получить чужой заказ, просто подставив другой id.
4. Безопасный запрос: Order.id + req.user.id.

Создание заказа: больше не доверяем userId из body

- Раньше учебный POST /api/orders мог принимать userId от клиента.
- После появления Authentication это больше не требуется.
- Владелец заказа определяется по req.user.id.
- Клиент передаёт только items.
- Так невозможно создать заказ от имени другого пользователя простой подменой JSON.

POST /api/orders с authenticated User

```javascript
export async function createOrderController(req, res, next) {
  try {
    const order = await createOrder(
      req.user.id,
      req.body.items
    );

    res.status(201).json(order);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- userId приходит из req.user.
- items приходят из validated body.
- Service получает доверенный id аутентифицированного пользователя.

Небезопасный и безопасный подход

Небезопасно

- body.userId определяет владельца
- Клиент может подменить id
- findUnique только по order.id
- UI скрывает чужие ссылки, но backend не проверяет

Безопаснее

- req.user.id определяет владельца
- JWT предварительно проверен
- Запрос учитывает order.id + userId
- Backend принимает окончательное решение

Учебный вопрос 5. Безопасная архитектура

Authorization — сквозное правило, а не одна проверка

Три уровня Authorization

Проверка доступа: маршрут (route) → бизнес-логика (service) → запрос к БД (database)

Пример: просмотр и удаление заказа. Каждый уровень проверяет права доступа.

Internet Shop
Secure API. Real Business.

Участники

Client
Отправляет HTTP-запрос с JWT (Bearer)

API (Express.js)
Проверяет доступ и обрабатывает запрос

Database (PostgreSQL)
Хранит заказы

1 Route: проверка роли

Определяем, кому доступен маршрут

```
import { requireAuth, requireRole }
 from './middlewares/auth.js';

// Роуты заказов
router.get(
  '/api/orders/:id',
  requireAuth,
  orderController.getById
);

router.delete(
  '/api/orders/:id',
  requireAuth,
  requireRole('admin'),
  orderController.deleteOrder
);
```

2 Service: владение ресурсом

Проверяем право на конкретный заказ

```
class OrderService {
  async getById(id, user) {
    const order = await prisma.order.findFirst({
      where: { id: Number(id) }
    });
    if (!order) throw new NotFoundError('Заказ не найден');

    // Доступ: свой заказ или администратор
    if (order.userId !== user.id && user.role !== 'admin')
      throw new ForbiddenError('Нет доступа к заказу');

    return order;
  }
  async deleteOrder(id) {
    // Удаление – только администратор
    return prisma.order.delete({ where: { id: Number(id) }});
  }
```

3 Database: запрос к БД

Выполняем безопасный запрос через ORM

```
// Получение заказа
await prisma.order.findFirst({
  where: { id: Number(id), userId: user.id }
});

// Удаление заказа
await prisma.order.delete({
  where: { id: Number(id) }
});
```

Сценарии работы

1. Просмотр заказа

GET /api/orders/:id

Client → GET /api/orders/15, Authorization: Bearer <JWT>

→ **Route**: Пользователь аутентифицирован

→ **Service**: Заказ принадлежит пользователю или он admin

→ **Database**: SELECT ... WHERE id = 15 AND userId = req.user.id

→ **200 OK**: Заказ возвращён

```
{
  "id": 15,
  "userId": 5,
  "status": "paid"
}
```

2. Удаление заказа

DELETE /api/orders/:id

Client → DELETE /api/orders/15, Authorization: Bearer <JWT>

→ **Route**: Проверена роль: требуется admin

→ **Service**: Выполняем удаление заказа

→ **Database**: DELEET FROM "order" WHERE id = 15

→ **204 No Content**: Заказ удалён

Возможные ответы API

- **200 OK** — Запрос выполнен (просмотр)
- **204 No Content** — Запрос выполнен (удаление)
- **401 Unauthorized** — Нет или неверный JWT (не выполнена аутентификация)
- **403 Forbidden** — Недостаточно прав (роль или владение ресурсом)
- **404 Not Found** — Заказ не найден

Authorization checklist

- [x] Deny by default
- [x] Least privilege
- [x] Проверять доступ на каждом защищённом запросе
- [x] Не доверять role из HTTP body
- [x] Не доверять userId клиента как identity
- [x] Проверять ownership конкретного объекта
- [x] Не считать скрытую кнопку защитой
- [x] Использовать 401 для отсутствующей Authentication
- [x] Использовать 403 при недостаточных правах
- [x] Логировать отказы доступа без утечки чувствительных данных

Authorization checks должны быть серверными

- Frontend может знать role для настройки интерфейса.
- Frontend может скрывать недоступные действия.
- Но любой frontend-код находится под контролем пользователя.
- Backend обязан повторно проверить role и ownership.
- Ни один UI-флаг вроде isAdmin не является доказательством права доступа.

Проверяйте каждый запрос

- Предыдущий успешный запрос не гарантирует право на следующий.
- Каждый endpoint должен самостоятельно проходить access control.
- Особенно важны create, update, delete и доступ к персональным объектам.
- OWASP рекомендует проверять разрешение для конкретного объекта или функции на каждом запросе.

Что логировать при отказе

- Можно логировать user id, endpoint, method и тип отказа.
- Не логируйте полный JWT.
- Не логируйте password или passwordHash.
- Authorization failures полезны для расследования атак и ошибок конфигурации.
- Логи сами должны быть защищены от несанкционированного доступа.

Типичные ошибки разграничения доступа

Ошибка	Риск	Исправление
role принимается из register body	Vertical privilege escalation	Назначать user на сервере
userId принимается из body	Действие от имени другого User	Использовать req.user.id
Проверяется только order.id	Доступ к чужому объекту	Проверять ownership
Кнопка скрыта только frontend	Прямой вызов API остаётся доступным	Server-side authorization
Admin route имеет только requireAuth	Любой logged-in User получает доступ	Добавить requireRole(admin)

Мини-практика: защита Internet Shop

Добавьте RBAC для Product API и ownership authorization для Order API.

Задание

- Создать requireRole()
- Оставить GET /api/products публичным
- Защитить POST/PATCH/DELETE Product ролью admin
- Создать GET /api/orders/my
- Использовать req.user.id вместо body.userId
- Запретить получение чужого Order
- Проверить 401, 403 и успешные сценарии

Критерии проверки

- User не может создавать или удалять Product
- Admin может управлять Product
- User видит только свои Order
- Подмена userId в body не влияет на владельца заказа
- Все access checks выполняются на backend

Полный access-control flow Internet Shop

От запроса клиента до защищённых данных

Internet Shop
Secure. Simple. For Everyone.

1. Client (Frontend)

Пользователь делает запрос



Запрос к защищённому API
с JWT в заголовке

```
GET /api/orders/15
HTTP/1.1
Host: shop.example.com
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6Ik...
```

2. requireAuth (Authentication)

Проверяем и декодируем JWT

```
const token = getToken(req);
try {
  req.user = jwt.verify(
    token,
    process.env.JWT_SECRET
  );
  next();
} catch (err) {
  return res.status(401).json({
    error: 'Unauthorized'
  });
}
```

req.user

```
req.user = {
  id: 5,
  email: "user@ex.com",
  role: "user"
}
```

401 Unauthorized

- Нет токена
- Неверный токен
- Истёк срок действия

3. requireRole / Authorization

Проверяем права доступа

```
function requireRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role) {
      return res.status(403).json({
        error: 'Forbidden'
      });
    }
    next();
};
```

Пример: только для администраторов

```
router.get('api/admin',
  requireAuth,
  requireRole('admin'),
  adminController.getUsers
);
```

403 Forbidden

- Пользователь аутентифицирован
- Но недостаточно прав (роль)

4. Controller

Обрабатываем запрос

```
async function getOrder(req, res) {
  const orderId = Number(req.params.id);
  const userId = req.user.id;

  const order = await orderService.
    getOrderById(orderId, userId);

  if (!order) {
    return res.status(404).json({
      error: 'Order not found'
    });
  }

  res.json(order);
}
```

404 Not Found

- Ресурс не найден
- Или не принадлежит пользователю
  (object-level authorization)

5. Service

Бизнес-логика и проверки владельца ресурса

```
async function getOrderById(id, userId) {
  // Запрашиваем только свои заказы
  return prisma.order.findFirst({
    where: {
      id: id,
      userId: userId
    },
    include: { items: true }
  });
```

6. Database

Возвращаем разрешённые данные

PostgreSQL (Prisma)

```
SELECT * FROM orders
WHERE id = 15
  AND user_id = 5;
```

Результат

```
{
  "id": 15,
  "userId": 5,
  "status": "paid",
  "total": 299.00,
  "items": [ ... ]
}
```

HTTP статусы

- **200 OK** — успешный запрос
- **401 Unauthorized** — нет/неверный JWT
- **403 Forbidden** — недостаточно прав
- **404 Not Found** — ресурс не найден (или не принадлежит)

Ключевые принципы

- Проверяйте подлинность (Authentication)
- Проверяйте права (Authorization)
- Проверяйте владение объектом (Object-level authorization)
- Не доверяйте данным от клиента (userId в body)
- Возвращайте корректные HTTP-статусы
- Логируйте попытки несанкционированного доступа

Роли в Internet Shop

- **admin** — Полный доступ (товары, заказы, пользователи)
- **user** — Доступ только к своим данным (свои заказы, профиль)
- **guest** — Доступ к публичным данным (каталог товаров)

Итог

Надёжный и гибкий механизм контроля доступа к данным Интернет-магазина.

- Безопасные данные
- Разделение ролей
- Защита на всех уровнях

Роль — только один из атрибутов доступа

- RBAC удобен для простых систем.
- Реальные правила могут учитывать владельца ресурса, статус заказа, организацию и другие атрибуты.
- Например User может отменить только свой Order со статусом new.
- Такое правило уже сочетает identity, ownership и состояние ресурса.
- Authorization следует моделировать по бизнес-правилам, а не только по ролям.

Контрольные вопросы

1. Чем Authorization отличается от Authentication?
2. Что означает deny by default?
3. Что означает least privilege?
4. В каком случае возвращается 401?
5. В каком случае возвращается 403?
6. Как работает requireRole()?
7. Почему нельзя принимать role из register body?
8. Почему userId владельца лучше брать из req.user?
9. Что такое object-level authorization?
10. Почему скрытая кнопка во frontend не является защитой?

Итоги занятия

1. Authentication подтверждает identity, Authorization проверяет право действия.
2. Безопасная политика строится по принципам deny by default и least privilege.
3. RBAC связывает разрешения с ролями user/admin.
4. requireRole() выполняется после requireAuth.
5. 401 означает отсутствие действительной Authentication, 403 — недостаточные права известного пользователя.
6. Frontend не является границей безопасности.
7. Admin CRUD endpoints должны проверяться на backend.
8. Object-level authorization проверяет доступ к конкретному ресурсу.
9. Владельца Order следует определять через req.user.id.
10. Каждый защищённый запрос должен проходить authorization check.